

Calcolatori Elettronici

9 settembre 2024

Teoria

Esercizio 1 (10 punti): Progettare una rete sequenziale in grado di interpretare sequenze infinite di valori binari a 3 bit. Ciascun valore viene inviata in input alla rete sequenziale un bit alla volta, partendo dal bit meno significativo. Dopo aver letto un valore, la rete sequenziale fornisce in output il valore "sì" se la cifra letta è dispari, in tutti gli altri casi fornisce in output il valore "no". Realizzare il circuito equivalente dell'equazione minimizzata di un flip flop di stato.

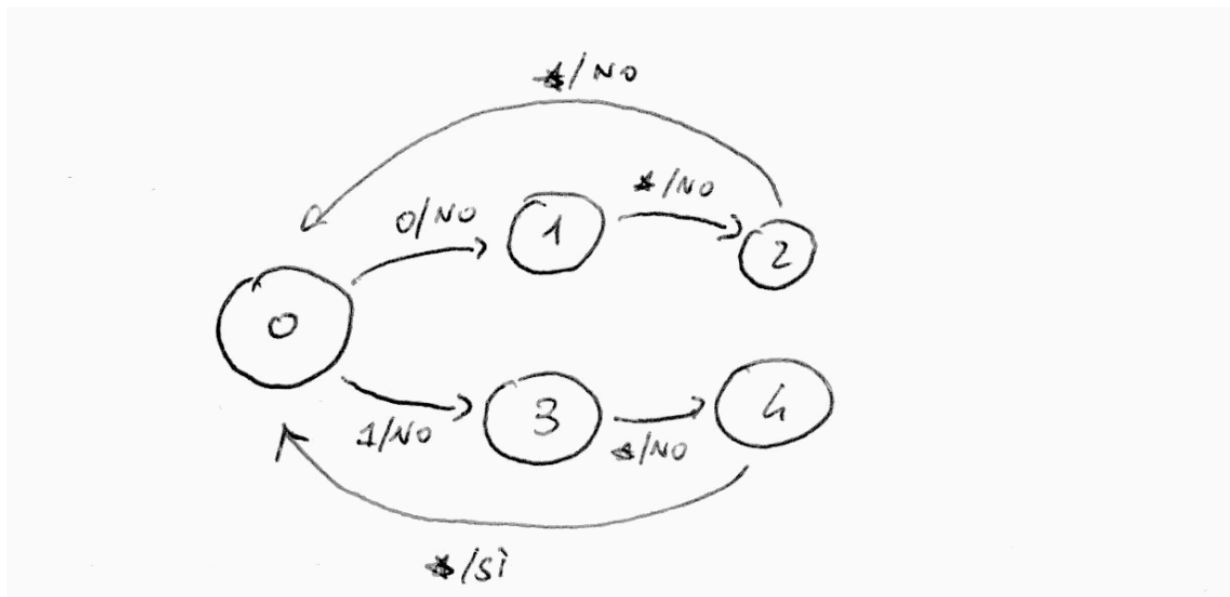
Esercizio 2 (10 punti): Codificare la parola binaria 0110110 utilizzando un codice di Hamming con $h = 3$, utilizzando bit di parità pari, mostrando tutti i passaggi. Si supponga di ricevere il valore 00001100111: illustrare come tale codifica permette di determinare la presenza di un errore di trasmissione.

Esercizio 3 (10 punti): Discutere il funzionamento del *carry lookahead adder*, illustrando per quale motivo tale implementazione di sommatore è preferibile rispetto ad un *full adder*, quando le parole binarie da sommare sono composte da un numero elevato di bit.

Soluzioni

Esercizio 1

Poiché il bit meno significativo è trasmesso per primo, la rete sequenziale decide dopo aver ricevuto un bit se l'output dopo aver ricevuto altri due bit sarà sì o no. L'automa a stati finiti che rappresenta tale rete sequenziale è quindi rappresentabile come segue:



La tabella degli stati e transizioni sarà:

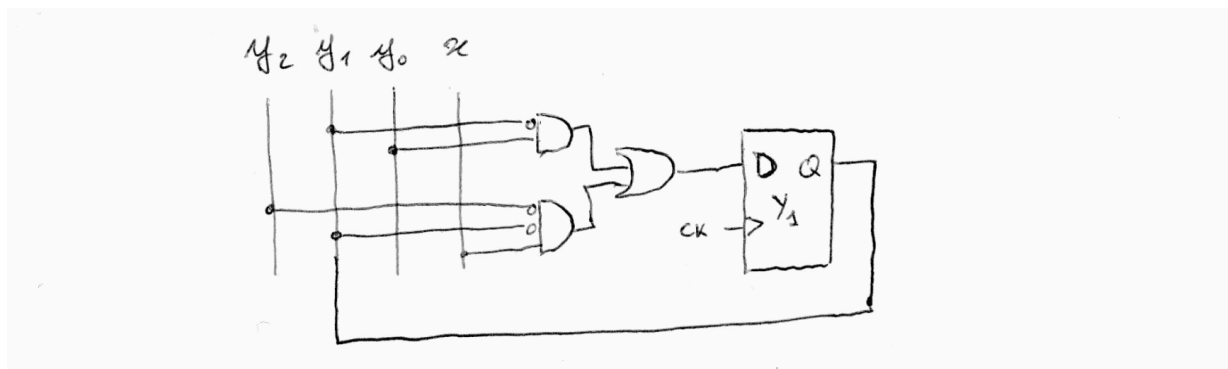
$y_2 y_1 y_0$	x	$y_2' y_1' y_0'$	z
000	0	000	0
000	1	011	0
001	0	010	0
001	1	010	0
010	0	000	0
010	1	000	0
100	0	000	1
100	1	000	1
011	0	100	0
011	1	100	0

in cui gli stati a partire da 101 sono don't care conditions. Volendo minimizzare y_1' , si può scrivere la seguente mappa di Karnaugh:

y_1'

$y_2 y_1$	y_0	00	01	11	10
00	x	0	0	-	0
01		1	0	-	0
11		1	0	-	-
10		1	0	-	-

che porta all'equazione di eccitazione $y_1' = \bar{y}_1 y_0 + \bar{y}_2 \bar{y}_1 x$ che può essere implementata dal circuito equivalente:



Esercizio 2

Per codificare la parola binaria 0110110 utilizzando un codice di Hamming con ($h = 3$) e bit di parità pari, occorre prima determinare la posizione dei bit di parità e poi calcolarne il valore. Un codice di Hamming aggiunge bit di parità in posizioni che sono potenze di 2 (1, 2, 4, 8, ...). I restanti bit sono i bit di informazione.

Dato che $h = 3$ ed il numero di bit da trasmettere è $n = 7$, dobbiamo aggiungere 4 bit di parità alla parola binaria di 7 bit. Questo porta a un totale di $m = 7 + 4 = 11$ bit (inclusi i bit di parità). Le posizioni dei bit di parità saranno 1, 2, 4 e 8. I bit di informazione verranno quindi inseriti nelle altre posizioni. Ordiniamo i bit come segue:

$P_1 \ P_2 \ b_1 \ P_3 \ b_2 \ b_3 \ b_4 \ P_4 \ b_5 \ b_6 \ b_7$

Dove P_1, P_2, P_3, P_4 sono i bit di parità e $b_1, b_2, b_3, b_4, b_5, b_6, b_7$ sono i bit di informazione. Inseriamo ora la parola binaria 0110110 nei bit di informazione:

$P_1 \ P_2 \ 0 \ P_3 \ 1 \ 1 \ 0 \ P_4 \ 1 \ 1 \ 0$

Ogni bit di parità controlla specifici bit dell'array, basandosi sulle loro posizioni binarie:

- P_1 controlla i bit nelle posizioni 1, 3, 5, 7, 9, 11 (ossia quelle con il primo bit in rappresentazione binaria uguale a 1): $P_1, b_1, b_2, b_4, b_5, b_7$. Questi sono: $P_1, 0, 1, 0, 1, 0$. Per parità pari, la somma deve essere pari, quindi $P_1 = 0$.
- P_2 controlla i bit nelle posizioni 2, 3, 6, 7, 10, 11 (ossia quelle con il secondo bit in rappresentazione binaria uguale a 1): $P_2, b_1, b_3, b_4, b_6, b_7$. Questi sono: $P_2, 0, 1, 0, 1, 0$. Per parità pari, la somma deve essere pari, quindi $P_2 = 0$.
- P_3 controlla i bit nelle posizioni 4, 5, 6, 7 (ossia quelle con il terzo bit in rappresentazione binaria uguale a 1): P_3, b_2, b_3, b_4 . Questi sono: $P_3, 1, 1, 0$. Per parità pari, la somma deve essere pari, quindi $P_3 = 0$.
- P_4 controlla i bit nelle posizioni 8, 9, 10, 11 (ossia quelle con il quarto bit in rappresentazione binaria uguale a 1): P_4, b_5, b_6, b_7 . Questi sono: $P_4, 1, 1, 0$. Per parità pari, la somma deve essere pari, quindi $P_4 = 0$.

Il codice Hamming risultante (inclusi i bit di parità) sarà quindi:

0 0 0 0 1 1 0 0 1 1 0

Supponiamo ora di ricevere il valore 00001100111. Determiniamo se c'è un errore di trasmissione utilizzando la decodifica del codice di Hamming:

- Verifica del bit di parità P_1 : controlla le posizioni 1, 3, 5, 7, 9, 11. I bit sono: 0, 0, 1, 0, 1, 1. La somma è dispari, quindi il bit di parità P_1 è errato.

- Verifica del bit di parità P_2 : controlla le posizioni 2, 3, 6, 7, 10, 11. I bit sono: 0, 0, 1, 0, 1, 1. La somma è dispari, quindi il bit di parità P_2 è errato.
- Verifica del bit di parità P_3 : controlla le posizioni 4, 5, 6, 7. I bit sono: 0, 1, 1, 0. La somma è pari, quindi il bit di parità P_3 è corretto.
- Verifica del bit di parità P_4 : controlla le posizioni 8, 9, 10, 11. I bit sono: 0, 1, 1, 1. La somma è dispari, quindi il bit di parità P_4 è errato.

La presenza di almeno un bit di parità errato indica che c'è stato almeno un errore di trasmissione. Si può provare a correggere un singolo errore di trasmissione, calcolando il numero di controllo $N_c = \langle P_1 P_2 P_3 P_4 \rangle = (1011)_2 = 11$, che ci indica la necessità di invertire il bit in posizione 11, permettendoci di ottenere:

00001100111 $\rightarrow$ 00001100110

Andando a rimuovere i bit di parità in posizione 1, 2, 4, 8, otteniamo 0110110, che era il valore inizialmente codificato.

Progetto

Un processore `z64` gestisce il sistema di controllo di un ponte levatorio. Il sistema comprende i seguenti dispositivi:

- `SENSORE_IN`: rileva in modo asincrono l'avvicinamento di una nave al ponte.
- `MOTORE`: consente di alzare e abbassare il ponte (attivato in modo sincrono).
- `SENSORE_OUT`: rileva in modo asincrono quando la nave ha superato il ponte.
- `SEMAFORO`: informa gli autoveicoli in arrivo dell'imminente apertura del ponte tramite segnali luminosi.
- `TIMER`: misura intervalli di tempo in secondi, secondo la programmazione del processore.

Il funzionamento del sistema è il seguente. Quando una nave viene rilevata da `SENSORE_IN`, lo `z64` programma `SEMAFORO`, in modo sincrono, per mostrare una luce rossa ed attiva `TIMER` per misurare un tempo pari a 120 secondi. Allo scadere dei 120 secondi, lo `z64` attiva `MOTORE` per sollevare il ponte. Quando la nave raggiunge `SENSORE_OUT`, `MOTORE` viene attivato per abbassare il ponte. Quando il ponte è abbassato, `SEMAFORO` mostra nuovamente una luce verde.

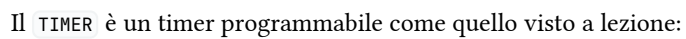
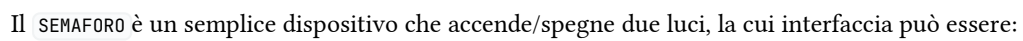
Si assuma che una sola nave alla volta può attraversare il ponte.

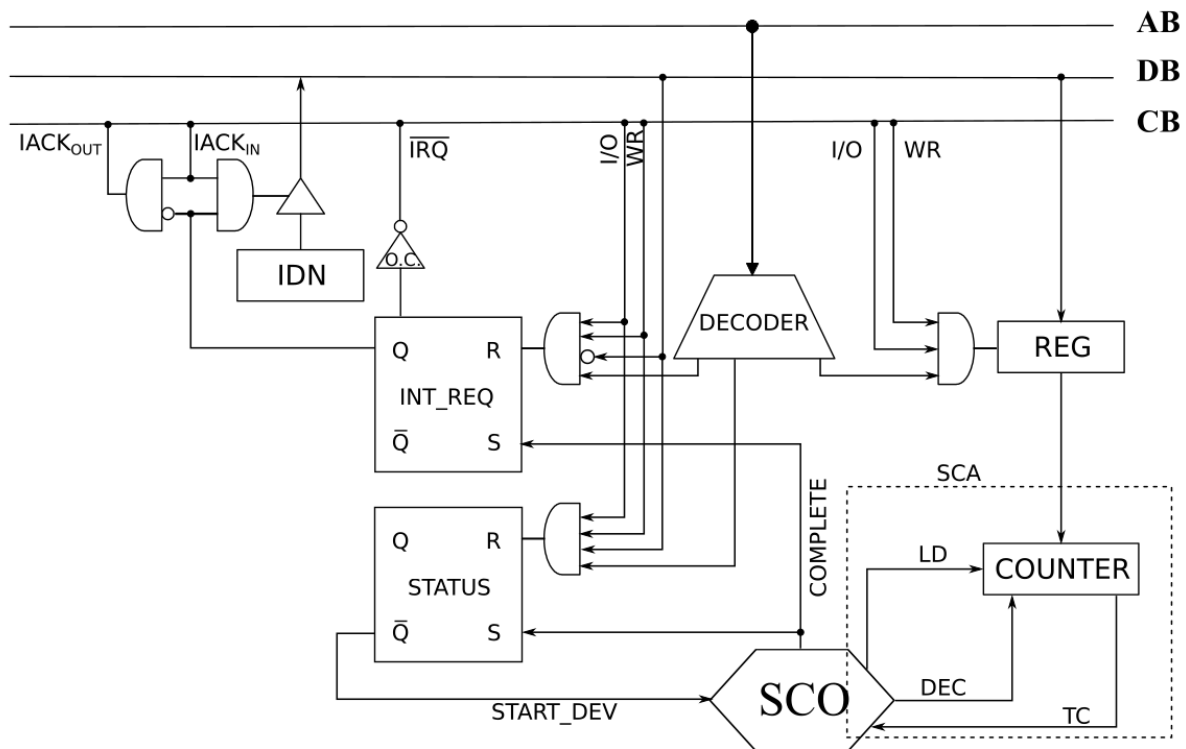
Realizzare:

- Le interfacce dei dispositivi `SENSORE_IN`, `SEMAFORO` e `TIMER` (non è necessario realizzare le interfacce di `SENSORE_OUT` e `MOTORE`);
- Tutto codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

Le interfacce richieste sono piuttosto standard. `SENSORE_IN` deve inviare richieste di interruzione quando rileva una nave, quindi può essere realizzata come segue:





Una possibile implementazione del software è la seguente.

```

1  .org 0x800
2  .data
3      .equ SENS_IN_IRQ, 0x00
4      .equ SENS_OUT_IRQ, 0x01
5      .equ MOTORE_STATUS, 0x02
6      .equ MOTORE_MODAL, 0x03
7      .equ SEMAFORO, 0x04
8      .equ TIMER_REG, 0x05
9      .equ TIMER_STATUS, 0x06
10     .equ TIMER_IRQ, 0x07
11     stato: .byte 0 # 0 = abbassato, 1 = alzato
12     azione: .byte 0 # 0 = deve essere abbassato, 1 = deve essere alzato
13     wait: .byte 1 # 0 = attesa trascorsa, 1 = attesa non trascorsa
14     .text
15
16 main:
17     sti
18     .loop:
19         movb stato, %al
20         cmpb azione, %al
21         jz .loop          # Se stato == azione, non c'è nulla da fare
22
23         cmpb $1, azione   # Controlla se si deve alzare o abbassare il ponte
24         jnz .lower_bridge
25
26         movb $0, %al      # Accende la luce verde
27         outb %al, $SEMAFORO
28         call commuta_ponte
29         jmp .loop
30
31     .lower_bridge:
32         movb $1, %al      # Accende la luce rossa
33         outb %al, $SEMAFORO
34
35         movb $120, %al    # Avvia un timer di 120 secondi

```

```

36     outb %al, $TIMER_REG
37     movb $1, wait
38
39     .loop2:                # Attende i 120 secondi
40     cmpb $0, wait
41     jnz .loop2
42
43     call commuta_ponte
44     jmp .loop
45
46 commuta_ponte:
47     movb azione, %al      # Carica il valore di azione (0 per abbassare, 1 per alzare)
48     outb %al, $MOTORE_MODAL
49     outb %al, $MOTORE_STATUS
50
51     .bw:
52     inb $MOTORE_STATUS, %al
53     btb $0, %al
54     jnc .bw
55     ret
56
57
58 .driver 0 # SENS_IN
59     movb $1, azione
60     outb %al, $SENS_IN_IRQ
61     iret
62
63 .driver 1 # SENS_OUT
64     movb $0, azione
65     outb %al, $SENS_OUT_IRQ
66     iret
67
68 .driver 2 # TIMER
69     movb $0, wait
70     outb %al, $TIMER_IRQ
71     iret

```

Prova di laboratorio

Si vuole realizzare un programma in C che implementi un algoritmo di compressione Run-Length Encoding (RLE), ottimizzando la compressione a livello di bit per rappresentare le ripetizioni nel modo più compatto possibile. La stringa da comprimere deve essere passata come argomento a riga di comando.

Il funzionamento dell'algoritmo di compressione è il seguente. Le sequenze ripetute di caratteri sono sostituite con il numero di ripetizioni e il carattere ripetuto. Ad esempio, la stringa "AAAAAAA" verrà compressa come "7A".

Successivamente, il programma deve ridurre ulteriormente l'uso della memoria memorizzando le ripetizioni a livello di bit, utilizzando il minor numero possibile di bit per rappresentare ciascun numero di ripetizioni. La rappresentazione finale di ogni coppia (ripetizioni, carattere) dovrà essere fatta utilizzando 4 bit per il numero di ripetizioni e 8 bit per il carattere ASCII. Ogni coppia sarà quindi compressa in un blocco di 12 bit. L'output finale viene mostrato a schermo in esadecimale.

Ad esempio, lanciando il programma come:

```
1 | ./rle AAAAAAABBBCCCCDD
```

le operazioni svolte saranno:

1. applicazione dell'algoritmo RLE alla stringa di input:

- o "AAAAAAA" → 7A (7 occorrenze del carattere 'A')
- o "BBB" → 3B (3 occorrenze del carattere 'B')
- o "CCCC" → 4C (4 occorrenze del carattere 'C')
- o "DD" → 2D (2 occorrenze del carattere 'D')
- o La stringa compressa diventa: "7A3B4C2D".

2. compressione dell'output:

- o "7A" diventa 0111 01000001
- o "3B" diventa 0011 01000010
- o "4C" diventa 0100 01000011
- o "2D" diventa 0010 01000100
- o L'output finale (in binario) sarà 0111 0100 0001 0011 0100 0010 0100 0100 0011 0010 0100 0100.

3. stampa dell'output: l'output verrà rappresentato in esadecimale, generando quindi 741342443244.

Soluzione

La seguente è una possibile soluzione all'esercizio.

```

1  #include <stdbool.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  #define MAX_RUN 15// 4 bit possono rappresentare al più 15 ripetizioni
7
8  typedef struct {// Struttura per restituire il risultato dell'encode
9      char character;
10     int occurrences;
11 } run_result;
12
13 // Codifica al massimo 15 caratteri consecutivi
14 run_result encode_run(const char *ptr)
15 {
16     run_result result = {.character = *ptr, .occurrences = 0};
17     while(*ptr) {
18         result.occurrences++;
19         if(++ptr != result.character || result.occurrences == MAX_RUN) {
20             break;
21         }
22     }
23     return result;
24 }
25
26 // Funzione per comprimere la stringa utilizzando l'algoritmo RLE con manipolazione di bit
27 unsigned char *compress_string(const char *ptr, int *index)
28 {
29     size_t len = strlen(ptr);
30     unsigned char *compressed = malloc(len * 2); // caso peggiore
31     bool fill_upper_half = true;
32
33     while (*ptr) {
34         run_result result = encode_run(ptr);
35
36         if (fill_upper_half) {
37             compressed[*index] = (result.occurrences << 4) | (result.character >> 4);
38             (*index)++;

```

```

39         compressed[*index] = result.character << 4;
40     } else {
41         compressed[*index] |= result.occurrences & 0xF;
42         (*index)++;
43         compressed[*index] = result.character;
44         (*index)++;
45     }
46
47     ptr += result.occurrences;
48     fill_upper_half = !fill_upper_half;
49 }
50
51 // Se siamo rimasti con un byte riempito a metà, incrementa index per completarlo con 4 bit a zero
52 if (!fill_upper_half)
53     (*index)++;
54
55 return compressed;
56 }
57
58 int main(int argc, char **argv)
59 {
60     int compressed_size = 0;
61
62     if(argc != 2) {
63         printf("Utilizzo: %s <stringa_da_comprimere>\n", argv[0]);
64         return 1;
65     }
66
67     unsigned char *compressed = compress_string(argv[1], &compressed_size);
68     printf("Stringa compressa in esadecimale: ");
69     for(int i = 0; i < compressed_size; i++) {
70         printf("%02X", compressed[i]);
71     }
72     printf("\n");
73
74     free(compressed);
75     return 0;
76 }

```

